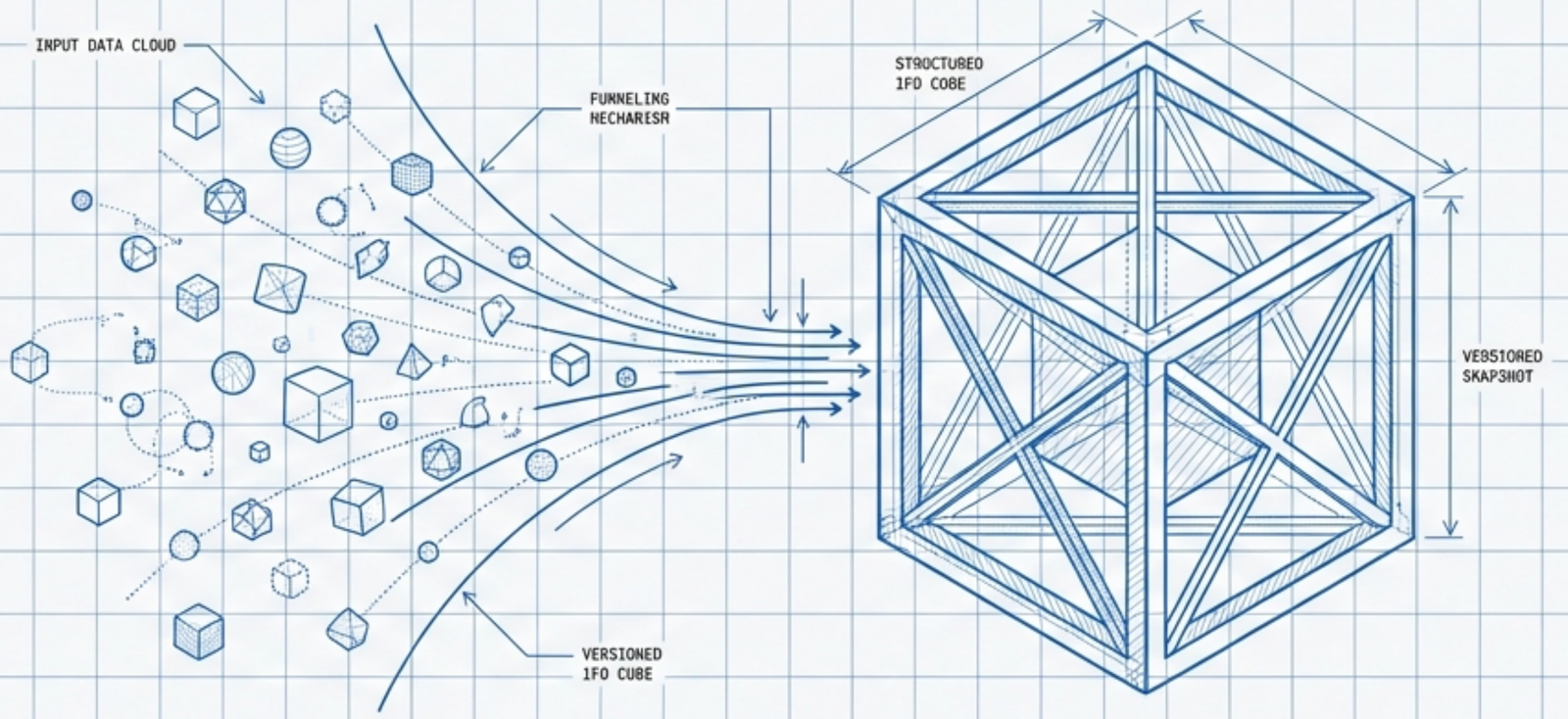
v1.4.53



IFD Snapshot Generator

Implementation Brief & Architectural Specification

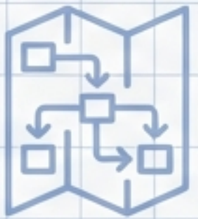
MODULE: mgraph_ai_service_html_graph.service.ifd_snapshot
DEPENDENCIES: Html_MGraph Service, OSBot-Utils v3.63+
STATUS: Ready for Implementation

Executive Summary

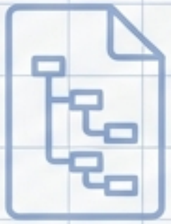
Programmatic Consolidation of Minor Versions

The IFD Snapshot Generator is a Python service designed to extract dependencies from an IFD minor version's HTML entry points to create a self-contained snapshot. It leverages the Html_MGraph multi-graph architecture to transform scattered development efforts into a consolidated release artifact.

The Deliverables

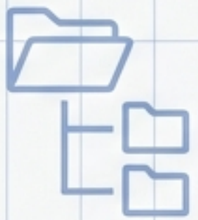


manifest.json: Complete dependency mapping with load order.



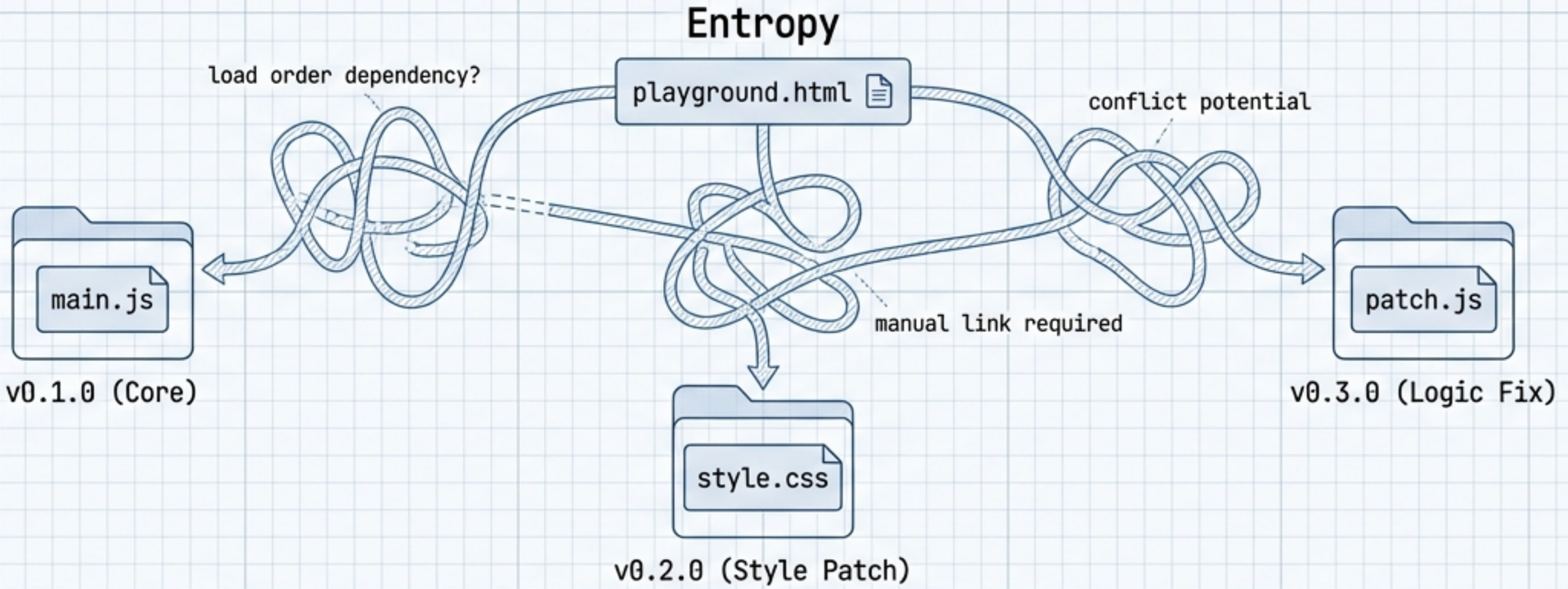
content.txt: Formatted text dump with tree structure and file contents.

snapshot.zip: In-memory zip generation (optional).



Physical Folder: Extracted contents for verification.

The Context:
The Surgical Override Pattern

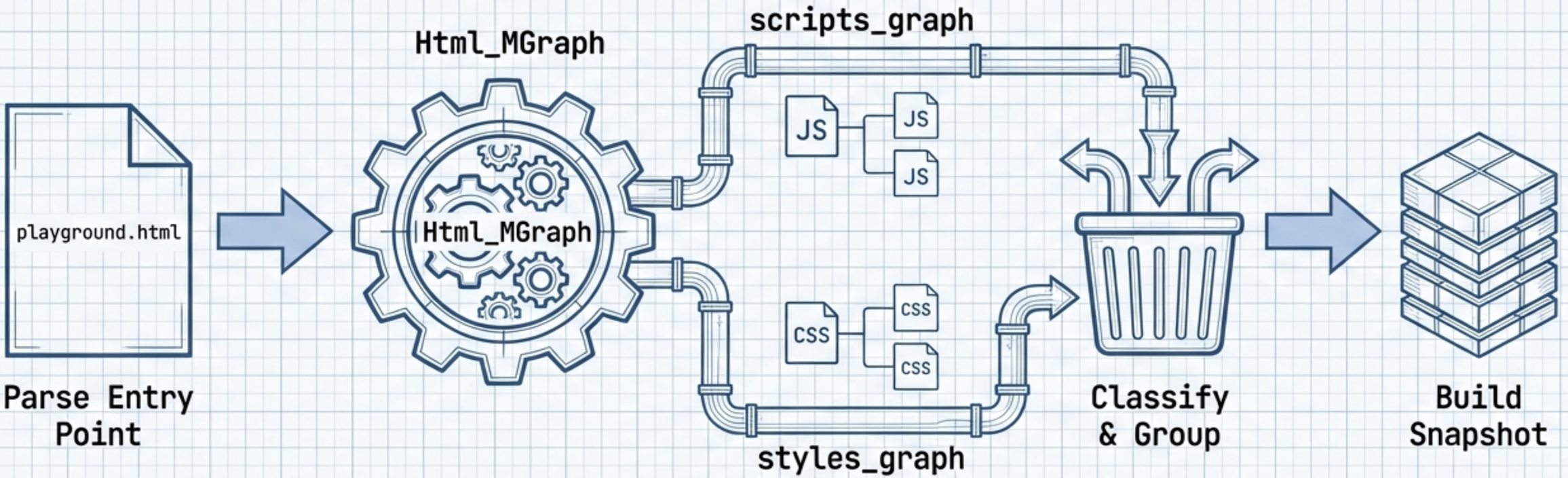


1. Manual consolidation is error-prone.

2. Must preserve exact load order.

3. Must track provenance (origin) of each file.

The Engine: Leveraging Html_MGraph Architecture



Key Insight: We do not parse raw HTML with regex. We utilize the existing Html_MGraph which already separates scripts and styles into dedicated graphs.

Data Foundation: Schema Definitions

Philosophy: Schemas are PURE DATA. No methods allowed.

Schema__IFD_Manifest.py
(Blueprint for snapshot)

Schema__IFD_Load_Chain.py
(Sequences dependencies)

Schema__IFD_Snapshot_Config.py
(Configuration parameters)

Schema__IFD_Dependency.py
(Defines single resource dependency)

Schema__IFD_Snapshot_Result.py
(Return object)

Schema__IFD_Html_Entry.py
(Represents entry point)

Safe_Types__IFD.py
(Custom safe type definitions)

Service Class Design

The Extractor

IFD_Dependency_Extractor.py

- Pulls nodes from the Html_MGraph
- Identifies dependencies via scripts_graph and styles_graph

The Builder

IFD_Snapshot_Builder.py

- Assembles the extracted data
- Constructs Manifest and Content Dump
- Handles Zip generation logic

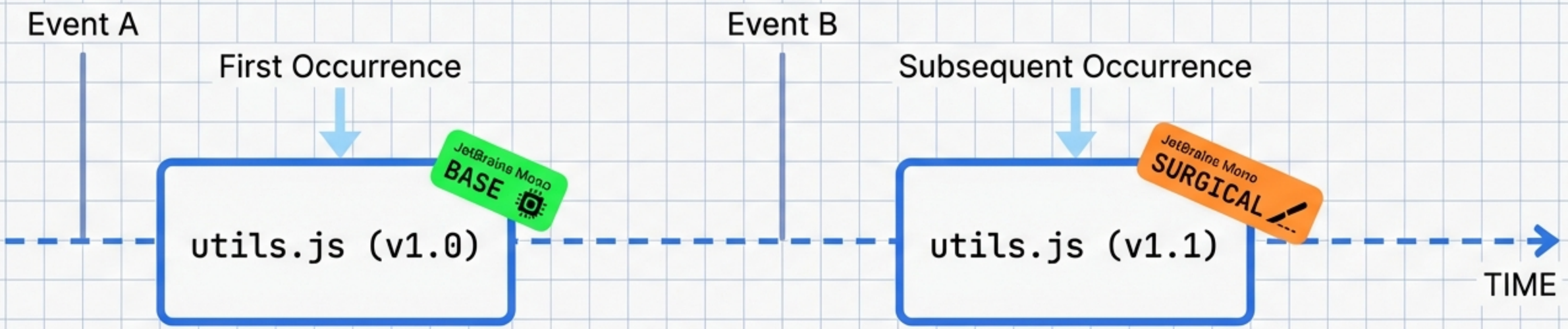
The Facade

IFD_Snapshot.py

- Public Interface
- Orchestrator
- Returns Schema__IFD_Snapshot_Result

Implementation Detail: Logic & Flow

Resolving Version Conflicts



Algorithm: The system determines "Base" vs. "Surgical" files based on the first occurrence of a logical name. Base files establish the foundation; Surgical files are treated as overrides or patches.

The Outputs (1): Manifest & Content

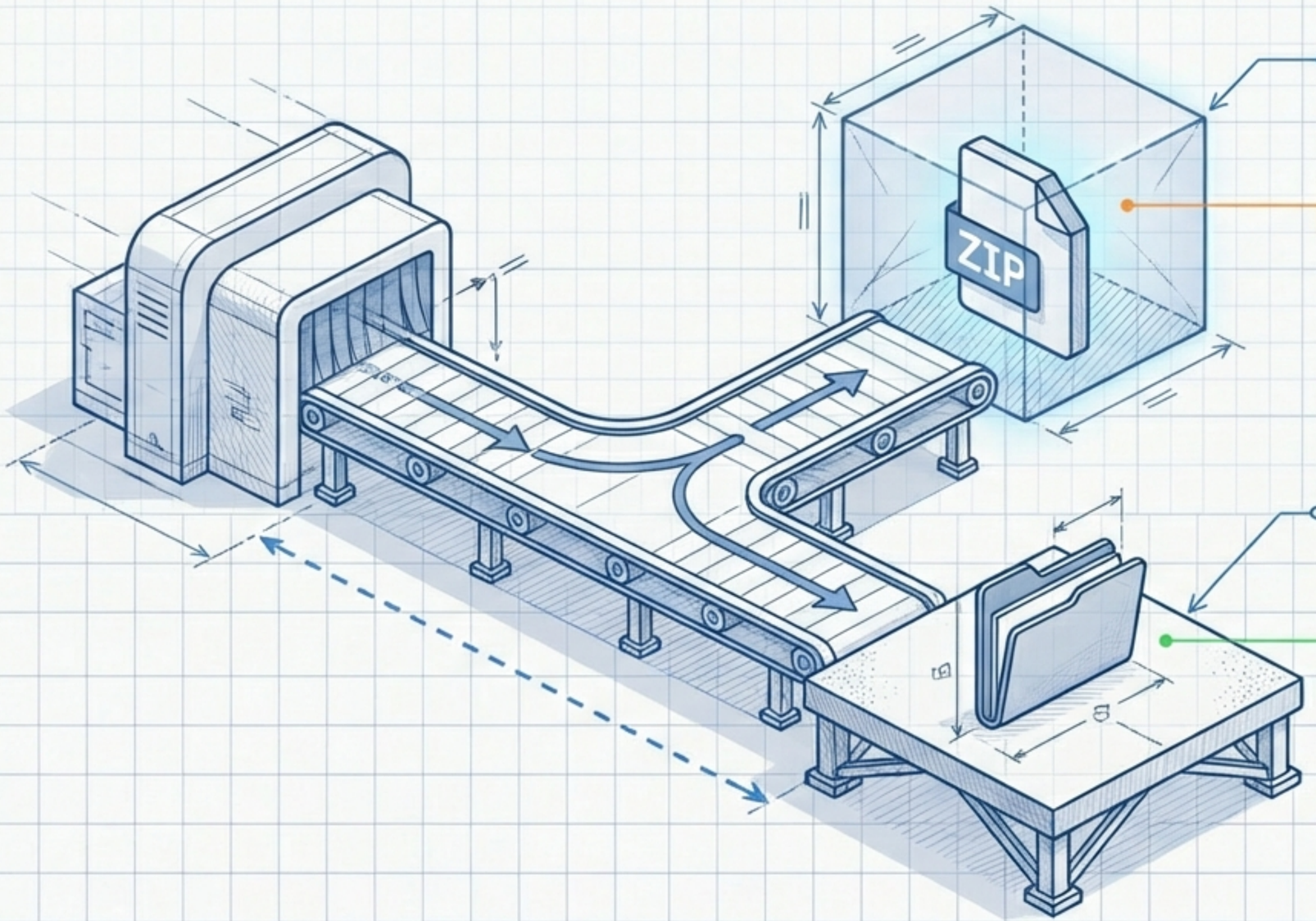
manifest.json (The Map)

```
{
  'files': [
    {'path': 'v1/main.js', 'type': 'script'},
    {'path': 'v2/style.css', 'type': 'style'}
  ],
  'load_order': [ ... ]
}
```

content.txt (The Territory)

```
├── v1/
│   └── main.js
├── v2/
│   └── style.css
----------------------------------------
FILE: v1/main.js
[...file content...]
```

The Outputs (2): Physical Artifacts



Memory

snapshot.zip (In-Memory)

Ideal for portability and direct download.

Disk

Extracted Folder (Physical)

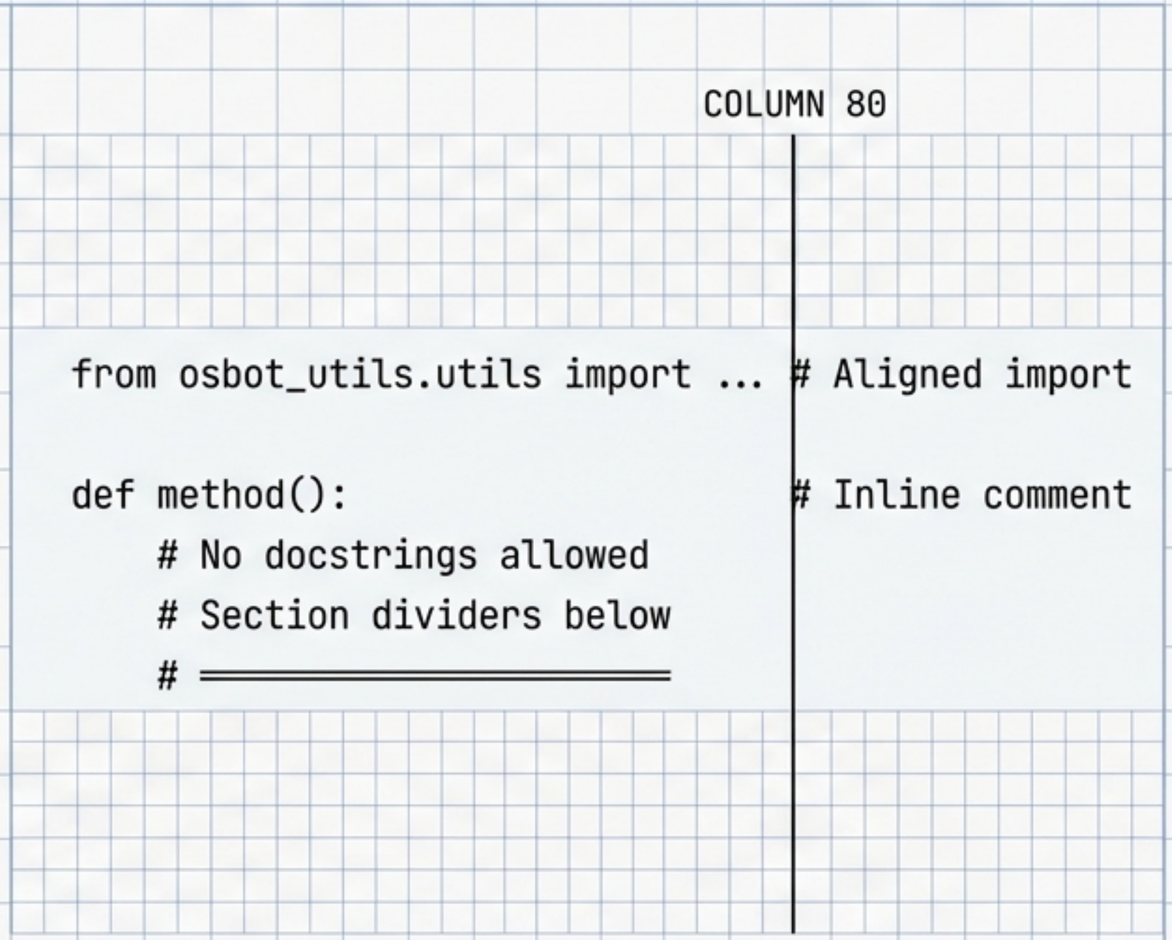
Reconstructed directory structure for verification.

PROJECT: IFD SNAPSHOT GENERATOR - SCHEMATIC V3.0

Critical Rules: Type Safety & Formatting

Type_Safe Mandate

- ✓ NO raw primitives. Use `Safe_Str`, `Safe_UInt`.
- ✓ Decorator: `@type_safe` on all methods.
- ✓ Context: Use ‘`with`’ pattern for robustness.



Critical Rules: The 'No Mocks' Strategy

Philosophy: Reality over Simulation.

Forbidden

- No @patch decorators
- No unittest.mock

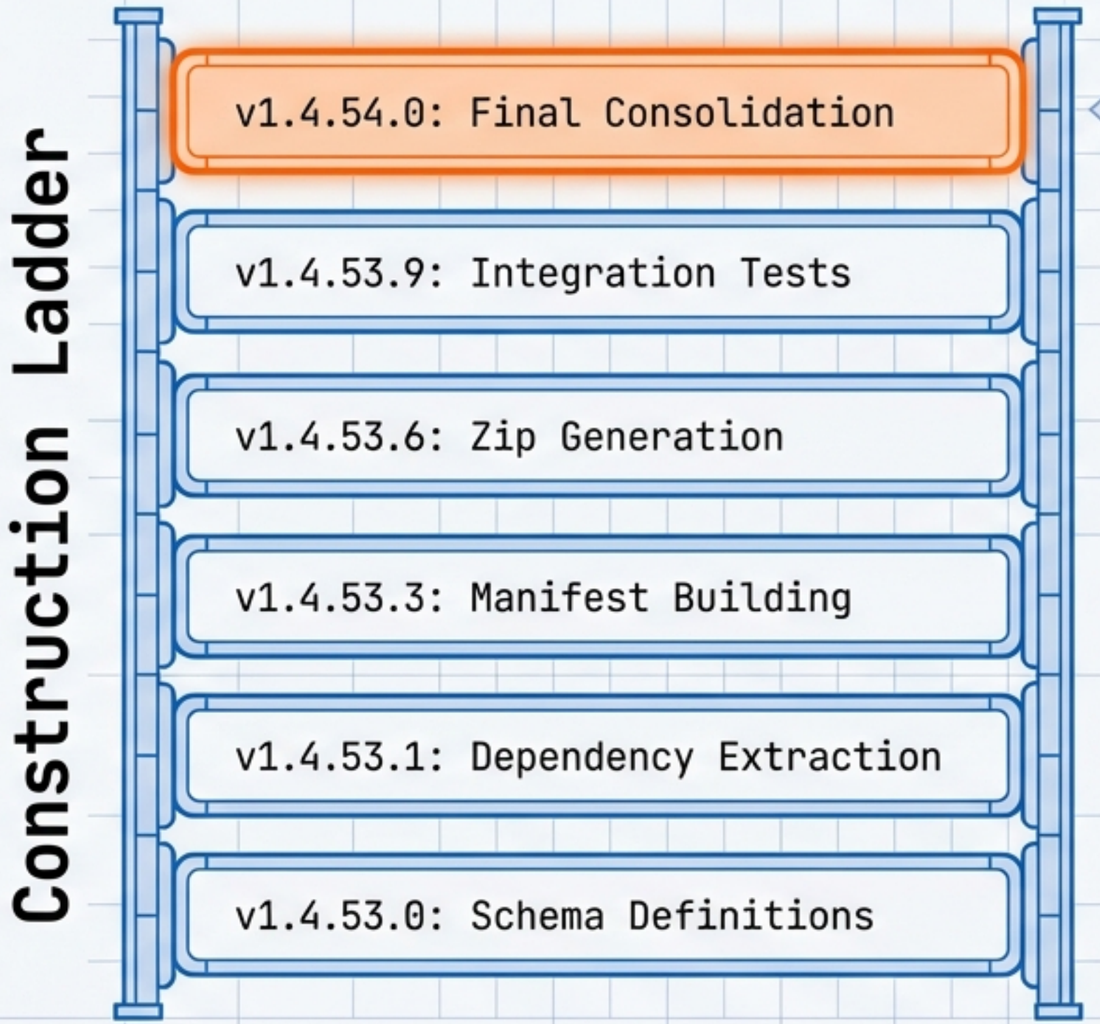
Required

- Real File Fixtures
- Real Html_MGraph Instances
- Context Managers

Key Tests

- tests/unit/.../test_IFD_Dependency_Extractor.py
- tests/integration/.../test_IFD_Snapshot__playground.py

Development Roadmap



MILESTONE

Methodology:
Progressive
Enhancement.

Usage Examples

Basic Usage

```
snapshot = IFD_Snapshot()
result   = snapshot.create(entry_file='playground.html')
```

Generate with Zip

```
result = snapshot.create(
    entry_file='playground.html', \
    create_zip=True
)
```

REST API Integration • Custom Source Folders

Summary of Capabilities

- ✓ **Programmatic:** Automates the consolidation workflow.
- ✓ **Deterministic:** Exact load order preservation via Html_MGraph.
- ✓ **Type-Safe:** Built on a foundation of Safe_Types.
- ✓ **Reliable:** Validated by a "No Mocks" testing strategy.

Status: v1.4.53 Ready for Implementation.

Built for reliable consolidation.